

Contents

1	An Unsaturated Coding-Agent Benchmark for Ruff	2
1.1	Abstract	2
1.2	1. Introduction	2
1.2.1	1.1 Motivation and Research Question	2
1.2.2	1.2 Contributions	2
1.3	2. Repository and Task Source	2
1.3.1	2.1 Repository Selection	2
1.3.2	2.2 Pull Request Sampling Window	2
1.3.3	2.3 Inclusion and Exclusion Criteria	3
1.3.4	2.4 Public Benchmark Decontamination	3
1.4	3. Benchmark Construction	3
1.4.1	3.1 Task Formulation	6
1.4.2	3.2 Candidate Collection	6
1.4.3	3.3 Task Prompt Construction	7
1.4.4	3.4 Production and Test Patch Separation	7
1.4.5	3.5 Static Filtering	7
1.4.6	3.6 Candidate Ranking and Triage	8
1.4.7	3.7 Final Task Selection	8
1.5	4. Dynamic Validation	8
1.5.1	4.1 Validation Criteria	8
1.5.2	4.2 Validation Environment	9
1.5.3	4.3 Validation Results	9
1.6	5. Agent Evaluation Methodology	9
1.6.1	5.1 Models	9
1.6.2	5.2 mini-swe-agent Configuration	9
1.6.3	5.3 Agent Inputs and Repository Access	9
1.6.4	5.4 Runtime, Step, and Cost Limits	9
1.6.5	5.5 Patch Submission and Extraction	10
1.7	6. Evaluator and Scoring	10
1.7.1	6.1 Patch Application and Tamper Checks	10
1.7.2	6.2 Compilation and Semantic Checks	10
1.7.3	6.3 Score Calculation	10
1.8	7. Results	10
1.8.1	7.1 Model Performance	10
1.8.2	7.2 Runtime, Cost, and Token Usage	11
1.8.3	7.3 Infrastructure Failures and Retries	11
1.9	8. Analysis	12
1.9.1	8.1 Performance by Subsystem and Task Size	12
1.9.2	8.2 Agent Failure Modes	12
1.9.3	8.3 Evidence of Benchmark Unsaturation	12
1.10	9. Limitations	12
1.11	10. Future Work	13
1.12	11. Reproducibility	13
1.12.1	11.1 Environment and Configuration	13
1.12.2	11.2 Reproduction Commands	14
1.12.3	11.3 Artifact Layout and Data Handling	16
1.13	12. Conclusion	17

1 An Unsaturated Coding-Agent Benchmark for Ruff

1.1 Abstract

This project develops a coding-agent benchmark from recent merged pull requests in [Ruff](#), a Python linter and formatter implemented primarily in Rust. The benchmark targets 100 automatically verifiable maintenance tasks. Each task is dynamically validated by requiring the relevant tests to pass at the base commit, evaluator-owned regression tests to fail before the fix, and those tests to pass after applying the original developer patch. Three models are evaluated using the mini-swe-agent harness in isolated environments, and their submitted patches are assessed by an automated scoring pipeline. On the completed 20-task sample, DeepSeek V4 Flash achieved the highest mean score (0.355), while GPT-5.4 mini produced the most patches and Qwen3 Codex had the lowest mean score (0.098). The results are promising for rapid benchmark construction but remain limited by the sampled task count and tight agent budgets.

1.2 1. Introduction

1.2.1 1.1 Motivation and Research Question

Evaluating model capabilities, including comparing models with one another, is important when selecting a model for a particular task. Some models may perform well on coding tasks, while others may be better suited to vision or other modalities. Existing coding-agent benchmarks such as SWE-bench provide a consistent way to measure coding performance, but new tasks are needed as models improve and existing benchmarks become less diagnostic.

Open-source projects are a useful source of new coding tasks because their issue descriptions, pull requests, developer patches, and regression tests provide evidence that a problem is both realistic and verifiable. This project explores whether those artifacts can be converted into a reproducible, automatically scored coding-agent benchmark: **can we generate a coding benchmark from an open-source repository whose tasks can be automatically validated and scored?**

1.2.2 1.2 Contributions

This project makes the following contributions:

1. A benchmark of 100 recent, real-world maintenance tasks mined from merged Ruff pull requests and screened against known public benchmark tasks, to ensure the same PRs in other benchmarks are not reused.
2. A reproducible construction and validation pipeline that separates developer solutions from evaluator-owned tests and verifies each task through base-pass, tests-only-fail, and gold-patch-pass outcomes.
3. An automated evaluator that checks submitted patches for compilation, hidden-test correctness, regression-test compilation, formatting, and evaluator-file tampering.
4. An empirical comparison of three coding models using a shared mini-swe-agent configuration, including correctness, runtime, and cost measurements.

1.3 2. Repository and Task Source

1.3.1 2.1 Repository Selection

In order to successfully choose a repository for this task, I wanted to choose a repository which was sufficiently large enough to mine enough verifiable tasks, but at the same time was not used extensively in existing benchmarks. This would allow for a large enough dataset to generate an eval set of desired size, while making sure the tasks are not in the model’s training data itself.

1.3.2 2.2 Pull Request Sampling Window

Pull requests were sampled from the [astral-sh/ruff](#) GitHub repository using a configured merge-date window of August 1, 2025 through July 29, 2026. The crawl excluded pull requests labeled `ty` at query time because

that subproject was outside the benchmark’s Ruff-core scope.

The resulting dataset contained 1,875 merged pull requests. Their observed merge timestamps ranged from August 28, 2025 through July 29, 2026. For each pull request, the crawler recorded its title, description, labels, base and head commits, changed files, and any explicitly linked issues.

1.3.3 2.3 Inclusion and Exclusion Criteria

The initial static filter retained a pull request only when it contained both a supported production change and a separable source of evaluator-owned tests. Supported production changes were Rust source files or the repository’s `Cargo.toml`, `Cargo.lock`, and `pyproject.toml` files. Test artifacts were identified from test, fixture, or snapshot paths; Rust test-file naming conventions; and snapshot files. Documentation, automation, and repository metadata could accompany an otherwise eligible change, but those files were not included in either the production or test patch.

Candidates were excluded when any of the following conditions applied:

- the pull request touched an out-of-scope path such as `playground/`, `ty/`, `crates/ty_`, or `crates/red_knot`;
- it lacked a supported production change or a separable test, fixture, or snapshot;
- it contained a changed path that could not be classified as production, test, ignored, or explicitly excluded;
- it changed fewer than 2 or more than 1,500 total lines, more than 30 files, or more than 20 production files;
- it carried a `dependencies` or `release` label, or its title indicated a release, dependency upgrade, or version bump;
- its production and test changes could not be split into two nonempty patches;
- its scrubbed task prompt was shorter than 40 or longer than 12,000 characters; or
- its pull request, base commit, prompt, or production patch overlapped a known public benchmark record.

A second static-triage stage inspected the separated patches more closely. It rejected candidates when the production and test patches modified the same file, when executable test registration remained in the production patch, when new fixtures or snapshots lacked an executable registration, or when no recognized test signal was present. The surviving candidates were ranked using test executability, subsystem, patch size, and the availability of a focused test command. Final inclusion additionally required the dynamic base-pass, tests-only-fail, and gold-patch-pass outcomes described in Section 4.

1.3.4 2.4 Public Benchmark Decontamination

Candidates were screened against SWE-Bench Multilingual’s `astral-sh/ruff` records (matched on PR number, base commit, prompt hash, and patch hash) and excluded on any match, removing 7 overlapping pull requests.

1.4 3. Benchmark Construction

Figure 1. Overview of the benchmark construction pipeline. Panel 1 shows how a single task is formed from a pull request’s description and (optional) linked issue, combined with the pre-change base commit, into a task package. Panel 2 shows the candidate selection pipeline from PR enumeration through deduplication, quality filters, and token-length filtering. Panel 3 lists representative exclusion and inclusion criteria applied during filtering. Panel 4 shows how the final, balanced task set is assembled and evaluated (model input, agent work, and evaluation).

Figure 2. Concrete example of the PR-to-task transformation. The selected PR supplies the natural-language problem, base commit, and production diff; candidate filtering and three-phase validation then establish that the change is self-contained, testable, and solvable. The final task exposes the problem and base repository to the model while retaining the gold patch and regression tests for evaluation.

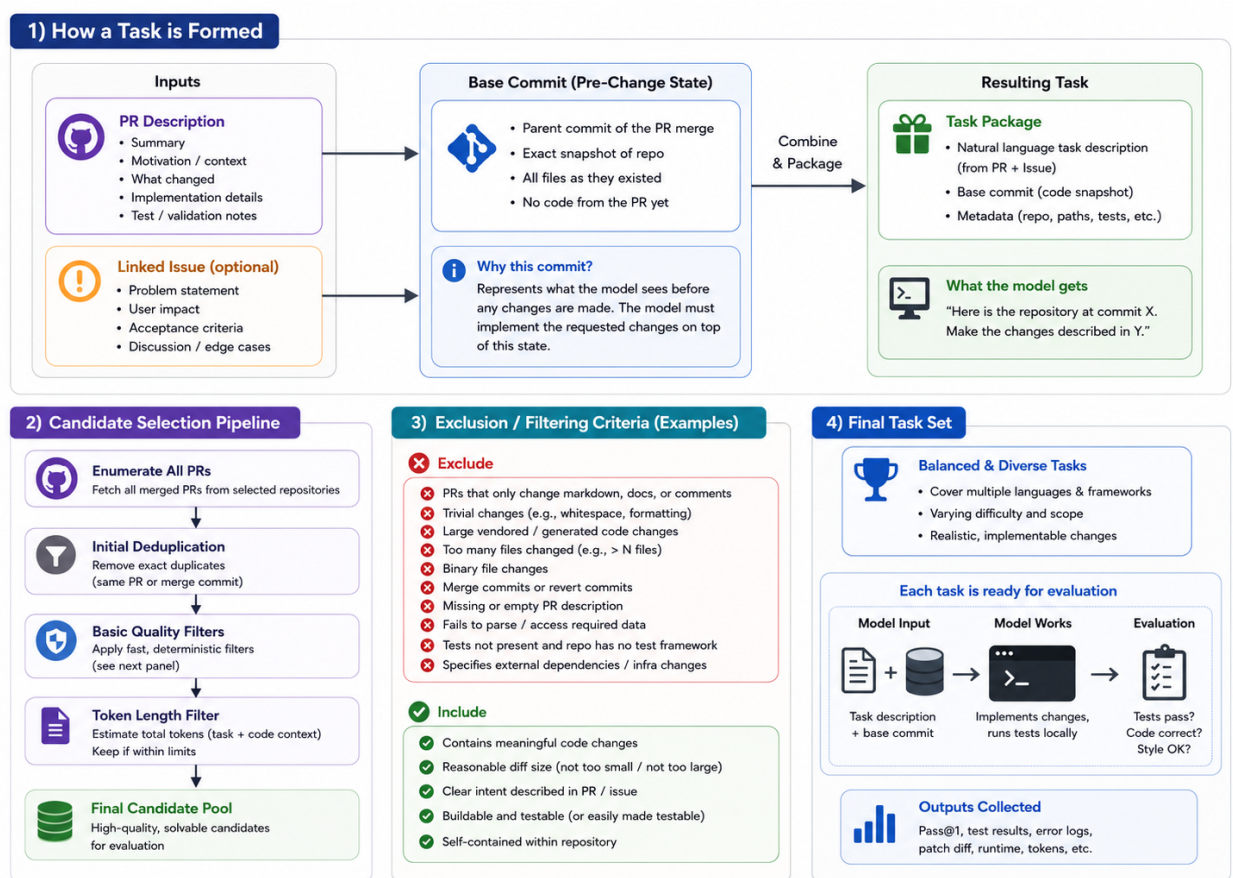


Figure 1: Figure 1. Benchmark construction pipeline: how a task is formed from a PR description and linked issue, the candidate selection pipeline, exclusion/filtering criteria, and the final task set.

Example PR → 1-Line Fix Task in MINI-UNSAT-SWE-BENCH

(Example from final 100-task benchmark)

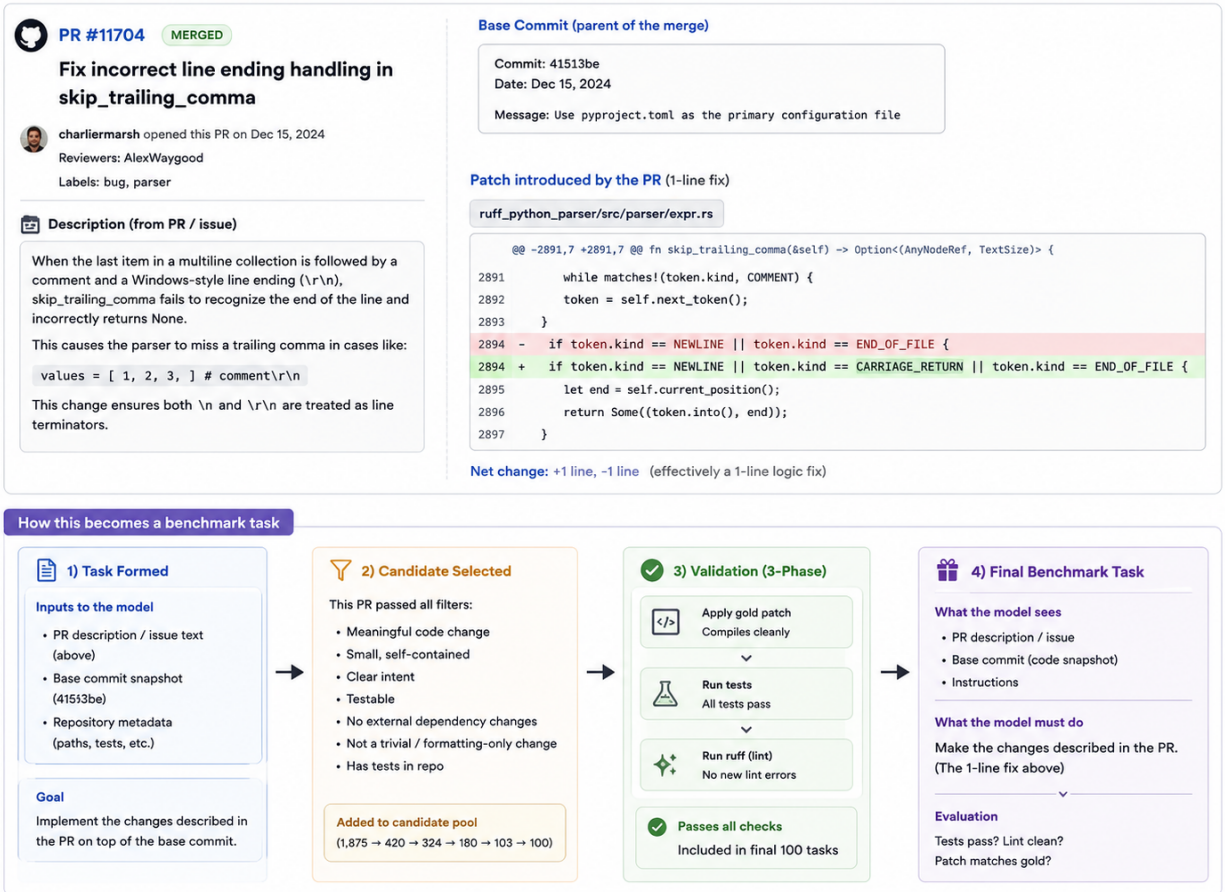


Figure 2: Figure 2. Example transformation of a merged Ruff pull request into a benchmark task.

1.4.1 3.1 Task Formulation

Each benchmark task asks an agent to modify Ruff at the state immediately before a selected pull request was applied. The agent is given the reported problem and the repository, but the artifacts used to validate and score its solution remain under evaluator control.

Agent receives

- └─ Natural-language task prompt
- └─ Ruff repository at the pull request's base commit
 - └─ Includes tests already present at that commit

Agent produces

- └─ A source-code patch

Evaluator adds

- └─ The model's patch
- └─ Hidden tests.patch

Evaluator runs

- └─ Compilation, hidden behavior, regression, and formatting checks

1.4.1.1 3.1.1 Agent Input The agent receives a natural-language prompt derived from the linked issue or pull request and the complete Ruff repository checked out at the pull request's base commit. The repository includes all source files and tests that existed at that commit. The agent can inspect files, edit source code, and run available commands through a terminal in an isolated container. It does not receive the new regression tests introduced by the resolving pull request.

1.4.1.2 3.1.2 Agent Output The agent must produce a unified Git diff containing its proposed source-code changes. The benchmark runner extracts this diff from the agent's final submission marker and saves it as `patch.diff`. The agent is not required to reproduce the original developer's implementation; any patch satisfying the evaluator is acceptable.

1.4.1.3 3.1.3 Evaluator-Only Artifacts Each task includes artifacts that are retained by the evaluator and are not provided to the agent:

- `tests.patch` contains the tests, fixtures, and snapshots introduced by the original pull request. These artifacts test whether a submitted patch fixes the targeted behavior.
- `gold.patch` contains the original developer's production-code changes. It is used to prove that the task is solvable during dynamic validation, but it is not applied when scoring a model submission.
- The effective test commands, scoring specification, validation results, and source metadata remain under evaluator control unless explicitly included in the scrubbed task prompt.

Separating these artifacts prevents an agent from copying the original solution or directly editing the hidden checks. It also allows alternative implementations to receive credit when they satisfy the same behavioral requirements.

1.4.1.4 3.1.4 Success Criterion A submission is fully resolved when its patch is nonempty, applies cleanly to the base repository, does not modify evaluator-owned paths, passes the compilation gate, and passes all hidden behavior, regression, and formatting checks. A fully resolved task receives a score of 1.0. The evaluator may award partial credit when the compilation gate passes but only a subset of the scored checks succeeds.

1.4.2 3.2 Candidate Collection

Candidate collection converted the fetched pull requests into task artifacts through the following procedure:

1. **Collect pull request metadata.** The crawler fetched 1,875 merged pull requests and recorded their descriptions, linked issues, commits, labels, and changed files.
2. **Classify the pull request.** The pipeline applied the path, change-size, label, title, and known-public-benchmark criteria from Section 2.3.
3. **Construct and check task artifacts.** For each survivor, the pipeline checked that the base and head commits were available, generated a scrubbed prompt, wrote production changes to `gold.patch`, and wrote tests, fixtures, and snapshots to `tests.patch`. It then applied the remaining prompt, nonempty-patch, and public-benchmark prompt/patch overlap criteria. Together, these two static steps rejected 1,455 pull requests and retained 420 candidates.
4. **Perform static triage.** Patch structure and executable-test evidence were inspected without running the code. This stage retained 324 eligible candidates, of which the 180 highest-ranked candidates formed the dynamic validation pool.
5. **Run dynamic validation.** Candidates were executed in Docker until more than 100 satisfied the base-pass, tests-only-fail, and gold-patch-pass requirements. Within the ranked pool, 103 candidates validated and 11 were rejected across 114 completed attempts.
6. **Freeze the benchmark.** The first 100 validated candidates in the deterministic ranking were written to the final task index. Model results were not used during task selection.

For example, `ruff_ruff-25641`, “Preserve whitespace for Quarto cell option comments,” modified one formatter source file and supplied an integration test, a formatter fixture, and a snapshot. The pipeline separated those files into nonempty production and test patches, and the candidate continued to triage and dynamic validation.

1.4.3 3.3 Task Prompt Construction

The prompt comes from one natural-language source, chosen by preference: the linked issue’s title and body if one exists, otherwise the pull request’s own title and body. An issue states the problem from a user’s perspective, independent of the eventual fix, so it is preferred when available. Of the 420 static candidates, 246 (59%) used a linked issue and 174 (41%) fell back to the PR description; the frozen 100 preserve roughly the same split (59/41).

The chosen text is then scrubbed: implementation-revealing sections (**Solution**, **Approach**, **Test Plan**, etc.) are removed, PR/commit URLs and `#123` references are replaced with generic placeholders, commit SHAs are redacted, and HTML comments are stripped. This keeps the prompt from leaking the reference implementation or the identity of the resolving PR. Prompts outside a 40–12,000 character range are rejected.

1.4.4 3.4 Production and Test Patch Separation

Every candidate’s changed files are partitioned into disjoint production and test paths. A path counts as a test artifact if it matches a configured test-directory marker, a Rust `_test/_tests` suffix, or a test-related extension (fixture/snapshot); otherwise it counts as production if it has a supported source extension or is one of a few auxiliary files (`Cargo.toml`, `Cargo.lock`, `pyproject.toml`). A path matching neither marks the whole candidate as unclassified and rejects it, rather than guessing the split.

`gold.patch` (production paths) and `tests.patch` (test paths) are then diffed independently from the same base→head commit range; both must be nonempty or the candidate is rejected. This is what lets `tests.patch` stay hidden from the agent while `gold.patch` proves the task solvable (Section 3.1.3).

1.4.5 3.5 Static Filtering

The initial static filter applied the size, path, label, prompt, and decontamination criteria described in Section 2.3. It operated only on pull request metadata and generated patches; it did not execute Ruff or determine whether the tests passed. A pull request was rejected if any exclusion reason was present. Because one pull request could violate multiple criteria, individual rejection-reason counts are not mutually exclusive.

As a simple rejection example, `ruff_ruff-27284`, “Update Rust crate `serde` to `v1.0.229`,” was excluded because it did not contain a separable test, fixture, or snapshot. The change therefore could not supply an evaluator-

owned behavioral check. Similarly, `ruff_ruff-26436`, “fix typos,” reached the prompt construction step but was rejected because its scrubbed prompt contained fewer than 40 characters. In contrast, `ruff_ruff-25641` contained both a supported Rust production change and separable test artifacts and therefore survived this stage.

1.4.6 3.6 Candidate Ranking and Triage

Static triage examined the contents of the separated patches to estimate whether the proposed hidden tests would be executable. It checked for registered Rust tests, existing or newly added fixtures, snapshots, formatter expectations, file overlap between the two patches, and test registration accidentally left in the production patch. It also proposed focused Cargo commands based on the affected crate, test target, module, and newly added test-function names.

Eligible candidates received a priority score favoring direct test registration, existing behavioral artifacts, small production patches, and predictable Ruff subsystems. Candidates were then sorted first by eligibility, then by decreasing priority score, and finally by task identifier. The top-ranked 180 eligible candidates formed the dynamic pool.

For example, `ruff_ruff-19045` contained useful fixtures and snapshots for three `flake8-gettext` rules, but the corresponding `#[test_case]` registrations remained in `gold.patch`. Without the gold patch, those tests would not be registered, so triage rejected the candidate with `test_registration_left_in_gold`. Another candidate, `ruff_ruff-20273`, contained only a newly created snapshot without an independent test registration and was rejected as `only_new_unregistered_test_artifacts`. By comparison, `ruff_ruff-25641` received a priority score of 110 and two focused commands: the Quarto CLI test and the formatter test suite.

1.4.7 3.7 Final Task Selection

Dynamic-validation outcomes were joined back to the deterministic triage ranking. The final index contains the first 100 records whose saved status was `validated`; all 100 entries in the frozen index have that status. Three additional validated candidates remained outside the primary set. This procedure made the final selection reproducible while avoiding selection based on whether any evaluated model could solve a task.

1.5 4. Dynamic Validation

1.5.1 4.1 Validation Criteria

Dynamic validation runs each candidate’s focused test command(s) in three clean phases against the pinned base commit: `base` (no patches), `tests_only` (`tests.patch` only), and `gold` (`tests.patch` then `gold.patch`). A candidate is `validated` only if all three phases produce the required outcome; each phase ran once per candidate (`repeats: 1`), with multi-repeat flakiness checks reserved for after task selection.

1.5.1.1 4.1.1 Base-Commit Test Outcome The tests must `pass` unmodified at the base commit — a health check that the suite isn’t already broken or flaky before any hidden test is added. Failure is rejected as `base_regression_failure` (5 of 114 attempts).

1.5.1.2 4.1.2 Tests-Only Outcome With only `tests.patch` applied, the tests must `fail`. This is the “before” half of fail-to-pass: it proves the hidden test actually exercises the unfixed bug rather than being vacuous or already satisfied. Failing this check is rejected as `hidden_tests_do_not_consistently_fail` (2 of 114 attempts).

1.5.1.3 4.1.3 Gold-Patch Outcome With `tests.patch` then `gold.patch` applied, the tests must `pass` — the “after” half of fail-to-pass, proving the task is solvable by the original fix. Rejected as `gold_patch_apply_failure` if the patch doesn’t apply (1 of 114) or `gold_does_not_consistently_pass` if it applies but tests still fail (9 of 114, the most common dynamic rejection reason).

1.5.2 4.2 Validation Environment

Each phase runs in a disposable, network-disabled Docker container (`--network none`) built from a per-base-commit image, smoke-tested before use. Patches are applied with `git apply --check` then `git apply` in the phase’s fixed order, against a read-only mount of the staged artifacts. Containers are capped at 4 CPUs / 8 GB memory with a per-command timeout (900s default); a timed-out or Docker-level failure is recorded as an `infrastructure_error`, distinct from a semantic `rejected` outcome, so host problems are never conflated with candidate failures.

1.5.3 4.3 Validation Results

The dynamic validator accepted `ruff__ruff-25641`: its focused commands passed on the unmodified base, failed after applying `tests.patch`, and passed after applying both `tests.patch` and `gold.patch`. This is the expected transition for an executable regression test that is repaired by the original developer solution.

In contrast, `ruff__ruff-20375`, “Add `analyze.string-imports-min-dots` to settings,” survived both static stages but was dynamically rejected. Its focused test command passed in the base, tests-only, and gold phases. Because the evaluator-owned test did not fail before the solution was applied, the candidate was classified as `hidden_tests_do_not_consistently_fail` and could not enter the benchmark.

1.6 5. Agent Evaluation Methodology

1.6.1 5.1 Models

Three models were evaluated, all routed through OpenRouter with a shared model configuration (`drop_params`, `parallel_tool_calls`, `max_tokens`: 4096):

- `qwen/qwen3-coder-next`
- `deepseek/deepseek-v4-flash`
- `openai/gpt-5.4-mini`

1.6.2 5.2 mini-swe-agent Configuration

All models were run with `mini-swe-agent v2.4.5` using a shared `mini.yaml` config: `agent_class`: `AutoSubmitInteractiveAgent` (Section 5.5) in a Docker environment, `interpreter`: `[bash, -c]`, and a fixed system/instance prompt telling the agent to make the smallest production-only edit, leave tests and evaluator files untouched, avoid full-workspace or `--release` builds, and submit by echoing a fixed marker followed by `cat`-ing its diff. Limits (step, cost, wall-time) are covered in Section 5.4.

1.6.3 5.3 Agent Inputs and Repository Access

Each task runs in its own Docker image, built once and reused across models: the Dockerfile fetches the Ruff repository at the task’s `base_commit` (shallow, detached-HEAD checkout) directly into `/testbed`, so the agent starts from the exact pre-change state with no setup step of its own. Images are smoke-tested before use and reused when their base-commit and precompile labels already match. Containers run network-isolated (`--network none`); required crates are fetched with `cargo fetch --locked` at build time.

Early runs showed that a cold Ruff build could consume most of a task’s step and wall-time budget before the agent made any progress. The fix was to precompile: the image build runs `cargo test --no-run` for the core crates (`ruff`, `ruff_linter`, `ruff_python_formatter`, `ruff_python_parser`) ahead of time, and agent/evaluator containers mount that image’s own compiled target directory directly instead of masking it with an empty volume. The system prompt also tells the agent not to run full-workspace or `--release` builds, since the precompiled targets make that unnecessary.

1.6.4 5.4 Runtime, Step, and Cost Limits

Each run is capped at 30 steps, \$1.00, and 180 seconds of wall time (with a 45-minute outer container timeout as a hard backstop). As either the step or time budget runs low (15 steps or 120 seconds remaining), the

chat template injects a warning telling the agent to stop exploring and submit immediately.

1.6.5 5.5 Patch Submission and Extraction

Normally, the agent stages its diff and submits by echoing a fixed marker followed by `cat`-ing the patch, which the runner extracts into `patch.diff`. In practice, some agents ran out of steps, cost, or time while still mid-edit and never reached that submission step, silently losing an otherwise usable fix. To prevent this, the agent runs as `AutoSubmitInteractiveAgent`: on a step, cost, or time-limit exception, it captures `git diff --` before the container is torn down and, if the tracked-file diff is nonempty, submits it directly with an `AutoSubmitted<LimitName>` exit status instead of discarding it.

1.7 6. Evaluator and Scoring

1.7.1 6.1 Patch Application and Tamper Checks

Scoring applies the model’s submitted patch to the base commit first, then applies `tests.patch` on top of it, both with `git apply --check` before `git apply`. If either fails, the run is a gate failure (`patch_not_applied`). Before applying anything, the submission is diffed against `tests.patch` at the file-path level; any overlap is flagged `evaluator_tampered` and recorded as a `policy_violation`, since it means the agent edited the hidden tests instead of the production code.

1.7.2 6.2 Compilation and Semantic Checks

One gate check must pass before any scored check runs: `cargo check --locked -p <crate>` for the crate touched by the task. Three scored checks then run, one per weighted group:

- **core** — the actual tests, fixtures, and snapshots from `tests.patch`, run via the task’s target test command(s): the same focused command validated to fail-before/pass-after the gold patch in Section 4.1. This is the only check drawn from the hidden PR-derived tests; it is the direct measure of whether the model’s patch fixes the targeted behavior.
- **regression** — `cargo test --locked -p <crate> --no-run`, a generic compile-only check (unrelated to `tests.patch`) confirming the crate’s existing test suite still compiles against the model’s change.
- **quality** — `cargo fmt --all -- --check`, a generic formatting check (also unrelated to `tests.patch`) confirming the patch is cleanly formatted.

1.7.3 6.3 Score Calculation

Each group has a fixed weight (core 0.80, regression 0.15, quality 0.05; the scorer also supports an `edge` group, currently unused). A group’s score is its fraction of passing checks within that group, and the raw score is the weighted sum across groups.

Any gate failure — a failed compile, a failed patch/`tests.patch` application, tampering, or a timeout — overrides everything and forces a score of 0.0. Otherwise, if the core check fails, the score is capped at 0.20 regardless of how the other groups scored, so a patch that doesn’t fix the targeted behavior can’t earn significant credit from formatting or compilation alone. A task is `fully_resolved` only at a score of exactly 1.0.

1.8 7. Results

1.8.1 7.1 Model Performance

The completed experiment contains 20 tasks for each of three models (60 model-task runs). Scores are the weighted evaluator scores defined in Section 6.3; `fully resolved` means an exact score of 1.0.

Model	Tasks	Mean score	Score sum	Fully resolved	Partial/non-resolved	Patch produced
DeepSeek V4 Flash	20	0.355	7.10	3 (15%)	17 (85%)	11 (55%)
GPT-5.4 mini	20	0.295	5.90	3 (15%)	17 (85%)	18 (90%)
Qwen3 Coder Next	20	0.098	1.95	1 (5%)	19 (95%)	3 (15%)
Overall	60	0.249	14.95	7 (11.7%)	53 (88.3%)	32 (53.3%)

Model	0.00	0.05	0.15	0.20	0.95	1.00
DeepSeek V4 Flash	11	0	2	0	4	3
GPT-5.4 mini	5	1	2	8	1	3
Qwen3 Coder Next	18	0	0	0	1	1

1.8.2 7.2 Runtime, Cost, and Token Usage

Model	Mean runtime (s)	Mean steps	Step-limit hits	Wall-time hits	API cost
DeepSeek V4 Flash	178.9	22.1	3/20 (15%)	16/20 (80%)	\$0.173
GPT-5.4 mini	50.5	15.8	1/20 (5%)	0/20 (0%)	\$0.547
Qwen3 Coder Next	103.5	28.7	15/20 (75%)	4/20 (20%)	\$0.847
Overall	111.0	22.2	19/60 (31.7%)	20/60 (33.3%)	\$1.567

Runtime and cost are inference-run measurements and exclude one-time Docker image preparation. Token counts are retained in each `run.json`; the experiment summary reports cost and steps because token accounting differs by provider.

1.8.3 7.3 Infrastructure Failures and Retries

Outcome category	Runs	Share	Interpretation
Submitted normally	22	36.7%	Agent submitted before a hard limit.
Auto-submitted at a limit	11	18.3%	A patch was preserved when the step or wall-time limit was reached.
Limit reached without a patch	27	45.0%	No usable diff was available at termination.
Infrastructure errors	0	0%	No run failed because of Docker or evaluator infrastructure.

The limit categories are operational outcomes, not quality labels: a run can produce a patch and still receive a low score. No retries were required for infrastructure failures in this batch.

1.9 8. Analysis

1.9.1 8.1 Performance by Subsystem and Task Size

DeepSeek V4 Flash achieved the highest mean score in this run (0.355), ahead of GPT-5.4 mini (0.295) and Qwen3 Coder Next (0.098). This ranking should be read as an observation about this 20-task sample rather than a general model ranking: all three models saw the same tasks, but the sample is small and each model was run only once per task. The task mix was dominated by linter changes (15 of 20 tasks), with only one parser, one configuration, two formatter, and one other-core task, so the result primarily measures performance on Ruff’s linter-oriented fixes.

The strongest model by mean score was also the slowest. DeepSeek averaged 178.9 seconds per task and reached the 180-second wall-time limit on 16 of 20 runs. GPT-5.4 mini averaged 50.5 seconds, while Qwen averaged 103.5 seconds. Thus, the higher DeepSeek score came with substantially longer trajectories; the result is not simply a speed-adjusted advantage.

1.9.2 8.2 Agent Failure Modes

The 30-step ceiling appears to be an important part of the observed result. Qwen used 28.7 steps on average and hit the step limit on 15 of 20 tasks; DeepSeek and GPT hit it on 3 and 1 tasks, respectively. Across all runs, 19/60 hit the step limit and 20/60 hit the wall-time limit. This concentration near the maximum means that some failures may reflect insufficient interaction budget rather than an inability to identify or implement the fix. A necessary follow-up is an ablation at larger limits (for example 60 and 120 steps, with correspondingly longer wall-time limits), while keeping the task set and scorer fixed. It would show whether additional turns convert partial patches into passing patches or merely produce longer unsuccessful trajectories.

The canary provides evidence that simply raising the limit may not solve the problem. In the precompiled three-model canary, Qwen3 Coder Next still reached the 75-step maximum without submitting a patch. This suggests a second hypothesis: some models spend too many turns exploring, repeating commands, or postponing the final edit. The benchmark should therefore report both budget ablations and a trajectory-efficiency metric (score per step or score per dollar). It may be more effective to evaluate models with stronger reasoning and planning that reach a correct patch in fewer turns than to keep increasing the maximum tool-call budget.

1.9.3 8.3 Evidence of Benchmark Unsaturation

The cost and score results show a useful trade-off. DeepSeek was both the highest-scoring model and the least expensive in aggregate (\$0.173 for 20 runs), compared with \$0.547 for GPT-5.4 mini and \$0.847 for Qwen3 Coder Next. However, DeepSeek’s low cost is partly a consequence of the provider’s token pricing and should not be interpreted as lower computational effort: its trajectories were the longest and frequently hit wall time. GPT produced the most patches (18/20) but many were incomplete, while Qwen produced only 3/20, which separates willingness to submit a diff from actually fixing the hidden tests.

The benchmark is not saturated by these results: no model resolved more than 15% of the sampled tasks, and 53/60 runs received a non-perfect score. That pattern is consistent with a challenging task set, but it is confounded by the tight interaction budgets and the small-model roster. Higher-step reruns, multiple independent samples, and at least one stronger reasoning model are needed before concluding that the remaining failures represent intrinsic task difficulty rather than trajectory inefficiency.

1.10 9. Limitations

Limitation	Consequence for interpretation	Mitigation / next step
Only 20 of 100 frozen tasks were evaluated	Results are a partial sample, not a full-benchmark resolution rate.	Run all 100 tasks, or report task-level confidence intervals for the sampled subset.
One run per model-task pair	Scores include stochastic variation and do not estimate pass@k.	Repeat selected tasks and report mean score and pass@k.
30-step and 180-second caps	Larger tasks may fail because the budget expired; this favors short fixes.	Run step/wall-time ablations, such as 30/60/120 steps.
Auto-submit at limits	Preserves work in progress but can evaluate incomplete patches and creates heterogeneous exit modes.	Report normal versus auto-submitted outcomes separately and compare with a higher-budget run.
Three relatively small/fast models	The experiment does not establish the capability ceiling of the benchmark.	Add a stronger frontier model under a separately reported cost budget.
Candidate validation used repeats: 1	A flaky test could be accepted or rejected incorrectly.	Repeat base, tests-only, and gold validation phases.
Prompts use only the first structured linked issue	Some maintainer context may be absent from the task prompt.	Resolve all linked issues and include a curated issue summary.
Small, PR-derived Ruff task sample	Results may not generalize to other repositories, task types, or languages.	Expand across repositories and add issue, documentation, configuration, and test-writing tasks.

1.11 10. Future Work

Step and wall-time ablations. Re-run the same tasks and models while varying `step_limit` and `wall_time_limit_seconds` (e.g., 30 vs. 60 vs. 120 steps) to measure how much of the resolution rate is capped by budget rather than model capability, and whether giving more steps closes the gap on larger tasks predicted by the step-limit bias in Section 9.

Broader model roster. Add one or more larger, more expensive models alongside the three small/fast models evaluated here. Because model choice was biased toward speed and cost, it is unclear whether low resolution rates reflect the benchmark’s difficulty or the roster’s capability ceiling; comparing against a stronger model would separate the two.

Multiple independent runs per task Rather than only 1 independent repeat per task, we can run multiple iterations of each task per model, and take the score as passed under some threshold (pass@k) or simply take the average. This reduces variance across model runs, which as presented in the results are only based on 1 independent build run due to time constraints.

1.12 11. Reproducibility

1.12.1 11.1 Environment and Configuration

The complete benchmark implementation, frozen task set, Dockerfile, agent configuration, model overlays, and scoring code are included in the submitted repository. Reproduction requires Python 3.11 or newer, Git, jq, and a running Docker Engine (Docker Desktop is sufficient on macOS). The experiments reported here used Linux containers with 4 CPUs and 8 GB of memory available to each agent container. Because Ruff compilation is memory- and storage-intensive, a host running two task workers should make at least 8 CPUs, 16 GB of memory, and sufficient Docker disk space available. Using fewer workers changes throughput, but not the tasks or scoring procedure.

From the repository root, create the pinned Python environment with:

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/pip install -r benchmark/requirements.txt
docker version
```

The second requirements file pins mini-swe-agent to version 2.4.5. Model API calls originate on the host, while every shell command issued by an agent runs inside a task-specific Docker container. The container has no network access (`--network none`), no Git remote, and no access to `tests.patch` or `gold.patch`. Dependencies are downloaded and the relevant Ruff targets are compiled while the image is built, before network isolation is applied.

OpenRouter authentication can be supplied either through the host environment or an ignored `.env.local` file:

```
export OPENROUTER_API_KEY=<key>
# Alternatively, put OPENROUTER_API_KEY=<key> in .env.local.
```

The batch runner reads `.env.local` automatically and does not copy the key into task images or containers. The frozen tasks are listed in `tasks/index.json`; shared agent limits and container settings are in `benchmark/mini.yaml`; experiment defaults are in `config.yaml`; and enabled models are registered in `benchmark/models.yaml`. These files should remain unchanged during a reportable comparison.

1.12.2 11.2 Reproduction Commands

The runner builds or reuses one image for each selected task, runs all selected models against fresh containers made from that image, extracts each submitted patch, evaluates it in another clean container, and then removes disposable resources. A one-task dry run can first be used to verify task and model selection without building an image or calling an API:

```
.venv/bin/python scripts/04_run_benchmark.py \
--run-id reproduction-check \
--task-id ruff__ruff-19571 \
--model deepseek-v4-flash \
--dry-run
```

Next, the same task can be run end to end. This is the quickest check that a model receives the prompt and pre-change codebase, produces a patch, and that the hidden-test scorer returns a result:

```
.venv/bin/python scripts/04_run_benchmark.py \
--run-id reproduction-smoke \
--task-id ruff__ruff-19571 \
--model deepseek-v4-flash \
--workers 1 \
--step-limit 20 \
--cost-limit 1.00 \
--wall-time-limit-seconds 180 \
--evaluation-timeout 300
```

To reproduce the reported 20-task, three-model experiment (60 model-task runs), load the predeclared task IDs from `benchmark/subsets/core20.txt` and run:

```
TASK_ARGUMENTS=()
while IFS= read -r task_id; do
  TASK_ARGUMENTS+=("--task-id "$task_id")
done < benchmark/subsets/core20.txt

.venv/bin/python scripts/04_run_benchmark.py \
--run-id reproduction-core20 \
```

```

--model qwen3-coder-next \
--model deepseek-v4-flash \
--model gpt-5.4-mini \
"${TASK_ARGUMENTS[@]}" \
--workers 2 \
--step-limit 30 \
--cost-limit 1.00 \
--wall-time-limit-seconds 180 \
--evaluation-timeout 300

```

```

.venv/bin/python scripts/06_summarize_results.py \
results/runs/reproduction-core20

```

Running the same command without the repeated `--task-id` arguments selects the complete frozen 100-task benchmark from `tasks/index.jsonl`. The `--workers` value controls only how many task images are processed concurrently; it can be reduced on a smaller host without changing benchmark semantics. If a run is interrupted, the same command can be resumed without repeating completed and scored model-task pairs by adding `--skip-existing`.

The runner evaluates patches immediately by default. If inference was deliberately collected with `--skip-evaluation`, it can instead be scored afterward with:

```

.venv/bin/python scripts/04b_evaluate_results.py \
results/runs/<run-id> \
--workers 2

```

```

.venv/bin/python scripts/06_summarize_results.py \
results/runs/<run-id>

```

The Docker environment can also be inspected independently of the agent runner. The following builds the image for one pinned task and executes its smoke test:

```

TASK_ID=ruff__ruff-20777
BASE_COMMIT="$(jq -r .base_commit "tasks/${TASK_ID}/task.json")"

```

```

docker build \
--file environment/Dockerfile \
--build-arg "RUFF_COMMIT=${BASE_COMMIT}" \
--build-arg PRECOMPILE=true \
--tag "mini-unsat-ruff:${TASK_ID}" \
environment

```

```

docker run --rm --network none \
"mini-unsat-ruff:${TASK_ID}" smoke-test

```

Precompilation makes the image build slower but prevents each model and evaluator container from paying the same cold Ruff compilation cost. For a quick environment diagnostic only, `PRECOMPILE=false` may be used; measured model comparisons should use the same precompilation setting for all tasks and models.

An existing registered model can be selected with one or more repeated `--model <id>` arguments. For example, the following runs only Qwen and DeepSeek:

```

.venv/bin/python scripts/04_run_benchmark.py \
--run-id selected-models \
--limit 20 \
--model qwen3-coder-next \
--model deepseek-v4-flash \
--workers 2

```

To add another OpenRouter model, create a model overlay such as `benchmark/models/my-model.yaml`:

```
model:
  model_class: openrouter
  model_name: provider/model-name
  cost_tracking: ignore_errors
  model_kwargs:
    drop_params: true
    parallel_tool_calls: true
    max_tokens: 4096
```

Then add an enabled entry to `benchmark/models.yaml`:

```
models:
- id: my-model
  config: benchmark/models/my-model.yaml
  api_key_env: OPENROUTER_API_KEY
  enabled: true
```

The model can then be selected with `--model my-model`. For a fair comparison, the shared prompt, task set, resource limits, step limit, wall-time limit, and cost limit must remain fixed. The exact provider-returned model identifier should also be retained because hosted aliases may change over time.

1.12.3 11.3 Artifact Layout and Data Handling

Each run is written under `results/runs/<run-id>/`. Its top-level `manifest.json` records the selected task and model IDs, mini-swe-agent version, limits, precompilation setting, and hashes of the task index, shared agent configuration, model registry and overlays, Dockerfile, and setup script. Each model-task directory contains:

```
results/runs/<run-id>/
├── manifest.json
├── runs.json
├── scores.json
├── summary.json
├── summary.md
├── subsystem-summary.json
├── pairwise-comparisons.json
├── <model-id>/<task-id>/
│   ├── environment.yaml
│   ├── trajectory.json
│   ├── runner.log
│   ├── patch.diff
│   ├── run.json
│   ├── eval.json
│   ├── checks.json
│   └── score.json
```

`trajectory.json` and `runner.log` preserve the agent interaction; `patch.diff` is the submitted model change; `checks.json` records evaluator outcomes; and `score.json` contains the final score and resolution status. Infrastructure failures are recorded separately from scorable model failures. API credentials and `.env.local` are excluded from the artifacts. Reproduction should retain the complete run directory and `manifest.json`, rather than only the aggregate summary, so every reported score can be traced back to its model trajectory, patch, environment, and individual evaluator checks.

1.13 12. Conclusion

This project was motivated by the need for recent, reproducible coding-agent tasks that remain useful as models improve and established benchmarks become less diagnostic. It investigated whether the artifacts already produced by open-source development—issue descriptions, pull requests, tests, and developer patches—could be converted into realistic tasks with automatic, objective scoring.

Ruff provided a large and actively maintained repository with substantial Rust code and a strong test suite. The construction pipeline collected recent merged pull requests, removed unsuitable changes and known public-benchmark overlaps, scrubbed implementation details from task prompts, and separated each developer solution from its associated tests. Candidates were then executed in isolated Docker environments. A task was accepted only when its tests passed at the base commit, the evaluator-owned tests failed without the fix, and those tests passed after applying the original developer patch. This process produced a frozen set of 100 validated tasks and three additional validated candidates. Further verification should repeat these validation phases to test for flakiness, and future benchmark runs should evaluate the remaining frozen tasks that were not included in the initial model experiment.

The initial evaluation demonstrates that the complete prompt-to-patch-to-score pipeline works and that the sampled tasks are not saturated by the tested models. Across the 20-task subset, DeepSeek V4 Flash achieved the highest mean score (0.355), GPT-5.4 mini produced the most patches, and Qwen3 Coder Next had the lowest mean score (0.098). No model fully resolved more than 15% of the sample, although many runs reached the step or wall-time limits, so these results should be treated as preliminary rather than as a definitive model ranking. Running all 100 tasks, repeating model-task trials, testing larger step and time budgets, and adding a stronger model would reduce sampling and budget uncertainty and provide a firmer measure of both model capability and benchmark difficulty.